Experiment 03: Interfacing I/O Peripherals

Roll Number: 21BIT175

TASK 1: Interfacing Output Peripherals

Objective: To interface LED to GPIO 4 of Raspberry Pi. Blink the LED every 1 second

Equipment and Components: Raspberry Pi – 4B, SD card/ USB Drive, Laptop, SD card reader (optional), breadboard, LED, resistors, male-female jumper wires

Theory: Interfacing output peripherals with a Raspberry Pi 4B is a fundamental aspect of leveraging the device's capabilities to interact with the physical world. The Raspberry Pi 4B, equipped with 40 General-Purpose Input/Output (GPIO) pins, serves as a versatile platform for connecting and controlling various output devices such as LEDs, motors, displays, and relays. These peripherals allow the Raspberry Pi to communicate with the external environment, enabling applications ranging from simple LED indicators to complex automation systems.

The GPIO pins on the Raspberry Pi 4B are highly flexible, capable of being configured as either input or output depending on the requirements of the connected peripheral. When set as outputs, these pins can be used to send electrical signals that control devices like LEDs, which can be turned on or off, or modulated to vary their brightness. Similarly, GPIO pins can control servo motors, allowing precise movement in robotics applications, or trigger relays to switch higher power circuits, making them indispensable in home automation projects.

In addition to basic peripherals, the Raspberry Pi 4B can interface with more sophisticated devices such as LCD displays, which require a combination of GPIO pins to manage data and control signals. By using libraries like RPi.GPIO or gpiozero, users can write Python scripts that send instructions to these peripherals, orchestrating their operation in real-time. This programmability is crucial for developing interactive systems, where the Raspberry Pi acts as the central controller, processing inputs and driving outputs based on logic and external conditions.

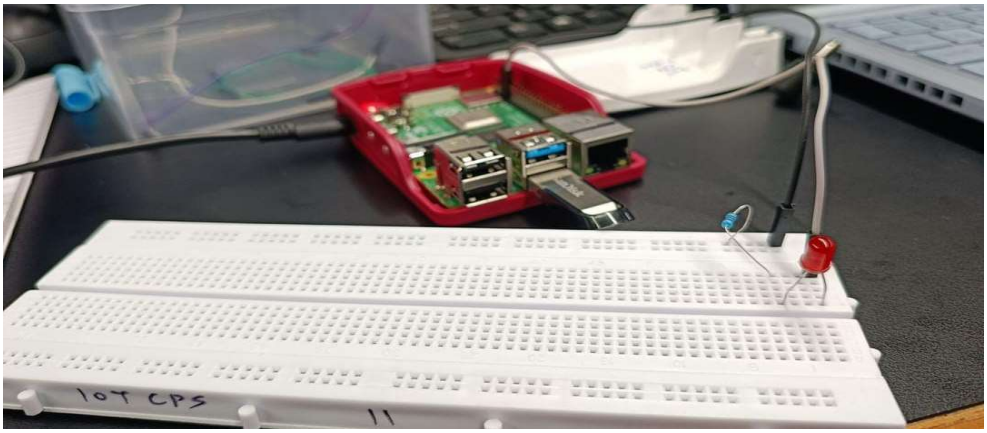
Furthermore, the Raspberry Pi 4B supports communication protocols such as I2C, SPI, and UART, which expand its ability to interface with a wider array of peripherals, including those that require high-speed data transfer or operate over longer distances. These protocols are essential when connecting devices like digital sensors, memory modules, or additional GPIO expanders, providing a scalable approach to building more complex systems.

Safety and power considerations are critical when interfacing output peripherals with the Raspberry Pi 4B. Since the GPIO pins operate at 3.3V, it is important to ensure that connected devices do not exceed this voltage, as doing so could damage the Raspberry Pi. Additionally, external power sources should be used for peripherals that require more current than the GPIO pins can safely provide. Proper use of resistors, transistors, and other protective components is necessary to prevent short circuits and overloading.

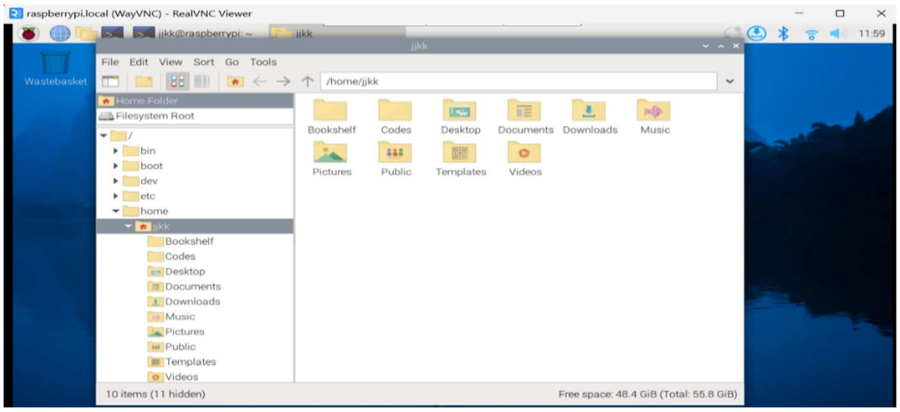
Procedure:

1. After setting up raspberry pi in experiment 2, now you can direct open PuTTY and repeat same steps as done previously. So, the Raspberry Pi OS desktop will appear.

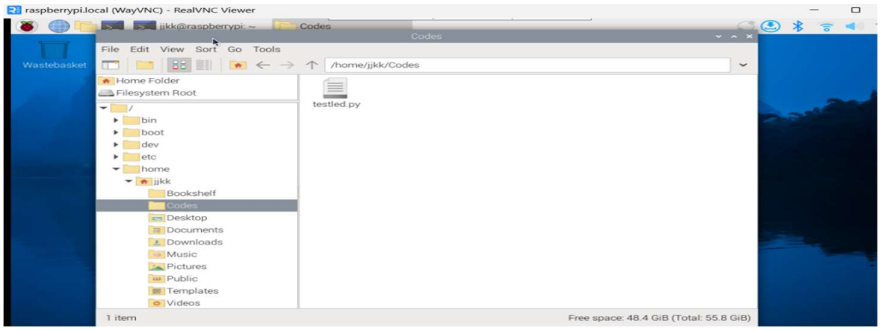
2. Then, make connections using breadboard, jumper wires, LED and RPi. Before doing this, check if the LED is in working condition or not. For doing this, connect LED to voltage supply pins of RPi. Now, proceed with making connections. Make sure you are taking GPIO 4 as an output pin. Here is the circuit.



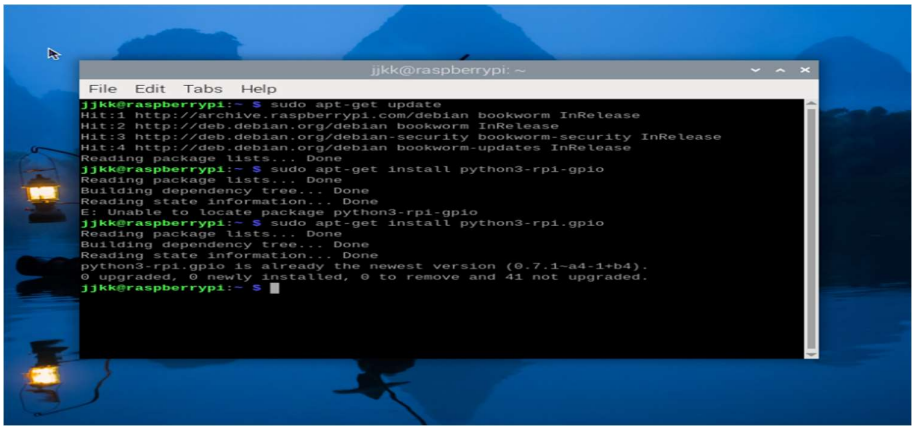
3. Next, click on File Manager icon on top menu bar. There you can create separate folder for our experiments. Here, we have created folder named ‘codes’.



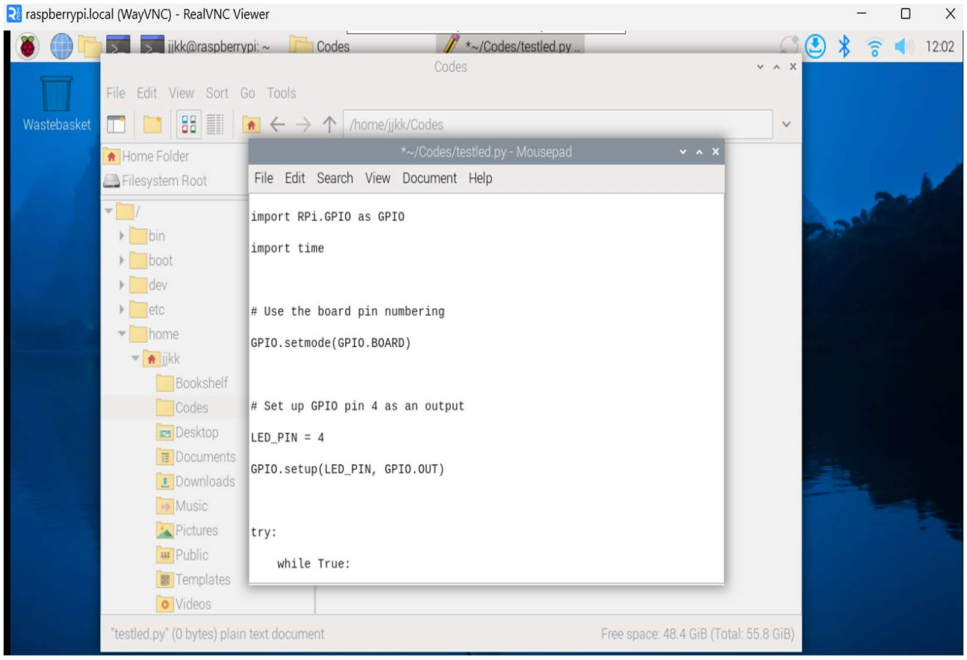
4. Now, create a file with .py extension in the folder that you created. Here, we have created file named testled.py



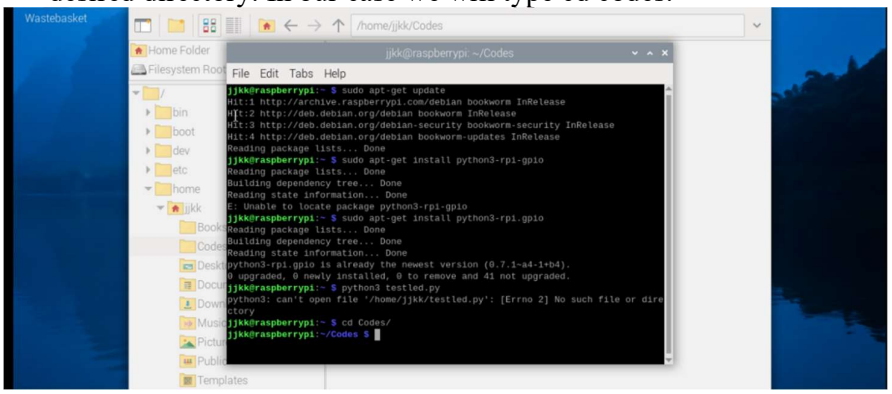
5. Then run following two commands in terminal:
   i. sudo apt-get update
   ii. sudo apt-get install python3-rpi.gpio



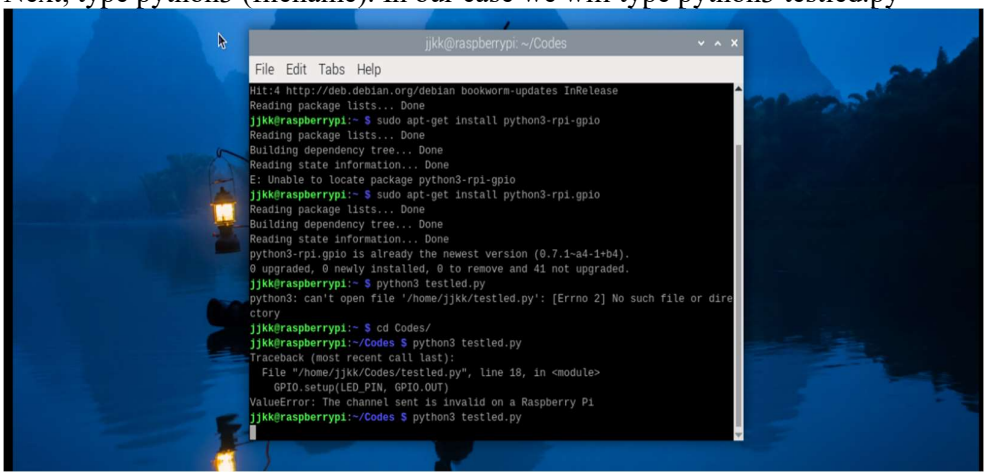
6. Further, write your python program in the file you created.



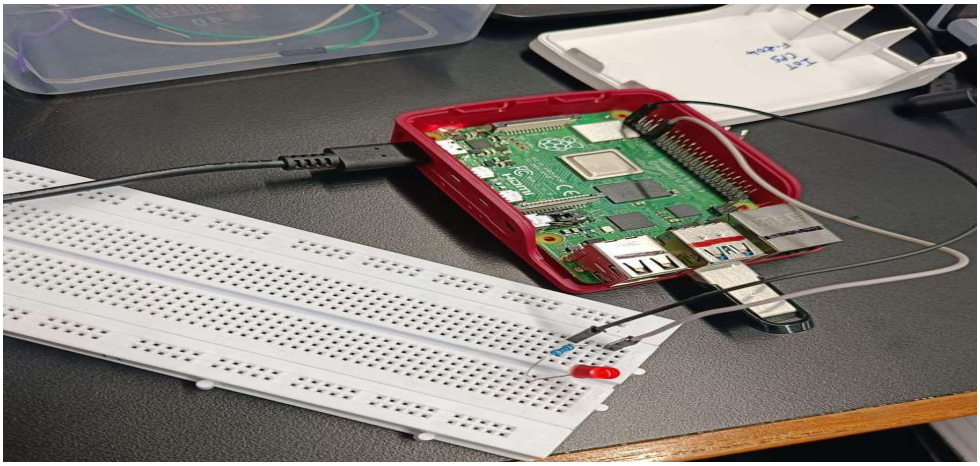
7. Then, to run your code, go to terminal and type **cd (**folder name**)**. This will take you to the desired directory. In our case we will type cd codes.



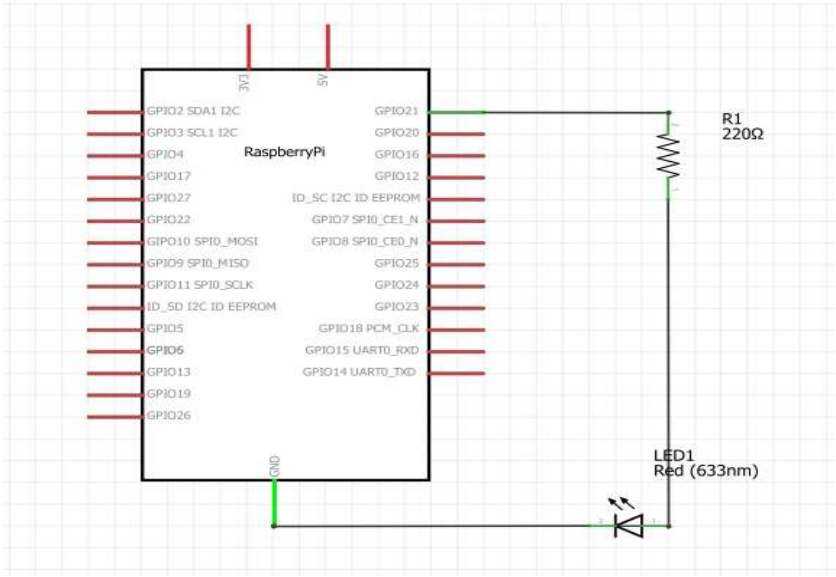
Next, type python3 (filename). In our case we will type python3 testled.py



This will start executing the code and LED will start blinking.



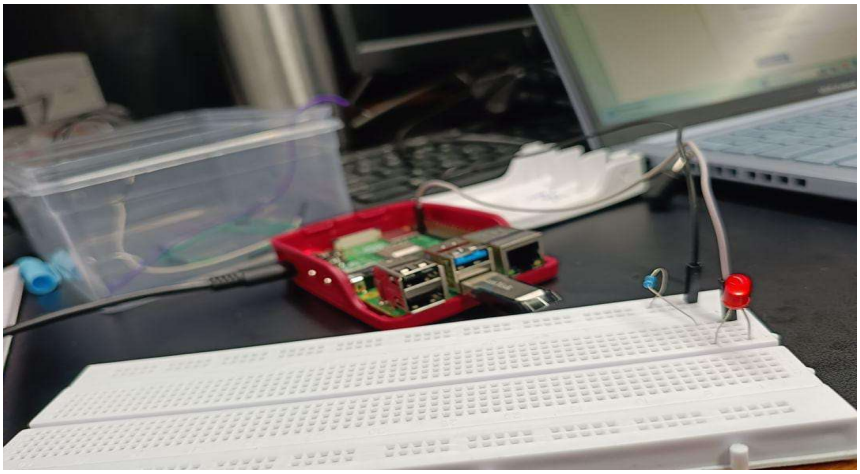
Block Diagram:



Code:

```
import RPi.GPIO as GPIO
import time

# Use the board pin numbering
GPIO.setmode(GPIO.BCM)

# Set up GPIO pin 4 as an output
LED_PIN = 4
GPIO.setup(LED_PIN, GPIO.OUT)

try:
    while True:
        # Turn the LED on
        GPIO.output(LED_PIN, GPIO.HIGH)
        time.sleep(0.5)  # Wait for 1 second

        # Turn the LED off
        GPIO.output(LED_PIN, GPIO.LOW)
        time.sleep(0.5)  # Wait for 1 second

except KeyboardInterrupt:
    # Cleanup GPIO settings before exiting
    GPIO.cleanup()
```

Results:



Discussion:

Conclusion:

TASK 2: Interfacing Input Peripherals

Objective: To control LED using a push button switch. When the switch is pressed, the LED should glow else it should not.

Equipment and Components: Raspberry Pi – 4B, SD card/ USB Drive, Laptop, SD card reader (optional), breadboard, LED, resistors, male-female jumper wires, button/switch

Theory: Interfacing input peripherals with the Raspberry Pi 4B is essential for enabling the device to receive and process data from the external environment. Input peripherals, such as buttons, sensors, and keyboards, allow the Raspberry Pi to gather information and respond to various physical conditions or user commands. These interactions form the basis for creating interactive systems, where the Raspberry Pi serves as the central unit, processing inputs and **executing corresponding actions.**

Switch / Button

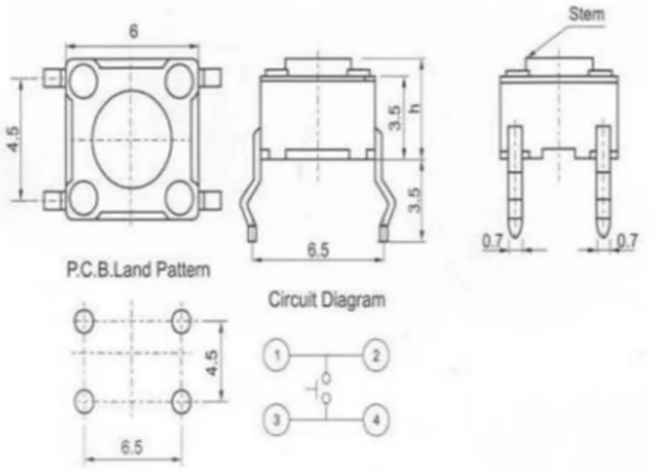
• Buttons are a common component used to control electronic devices.

• They are usually used as switches to connect or disconnect circuits.

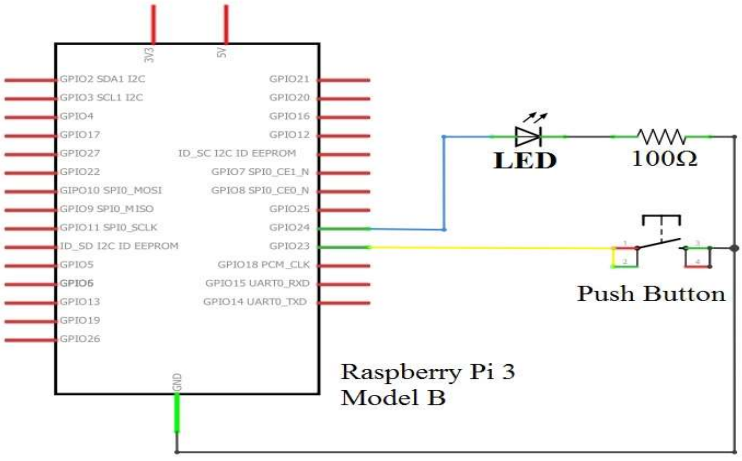
• Although buttons come in a variety of sizes and shapes, the one used here is a 6mm mini-button as shown in the following pictures.

• Pins pointed out by the arrows of same color are meant to be connected.

When the button is pressed, the pins pointed by the blue arrow will connect to the pins pointed by the red arrow (see the above figure), thus closing the circuit, as shown in the following diagrams.



Block Diagram:



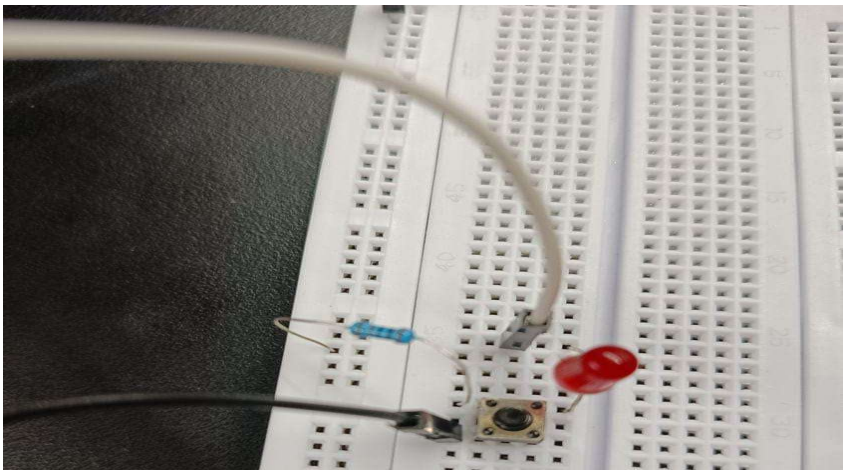
Code:

```
import RPi.GPIO as GPIO
import time


# Set up the GPIO mode
 GPIO.setmode(GPIO.BCM)


# Define the GPIO pins for the LED and the button
 LED_PIN =  27
 BUTTON_PIN = 22


# Set up the GPIO pins
 GPIO.setup(LED_PIN, GPIO.OUT)
 GPIO.setup(BUTTON_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)  # Use an internal pull-up resistor


try:
    while True:
        # Read the state of the button
        button_state = GPIO.input(BUTTON_PIN)
        if button_state == GPIO.LOW:  # Button is pressed
            GPIO.output(LED_PIN, GPIO.HIGH)  # Turn on LED
```

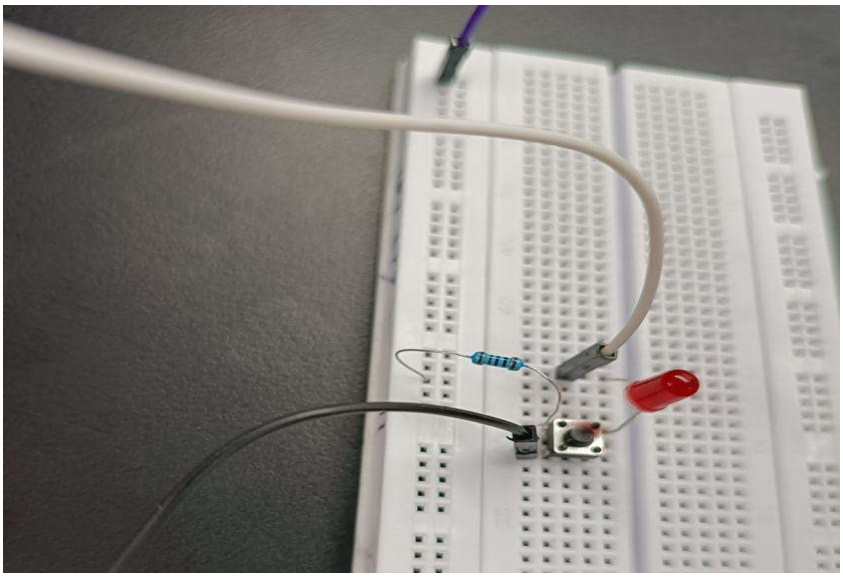
```
        else:
            GPIO.output(LED_PIN, GPIO.LOW)  # Turn off LED
        time.sleep(0.1)  # Small delay to debounce the button

except KeyboardInterrupt:

    # Clean up GPIO settings before exiting
    GPIO.cleanup()
```

Results:

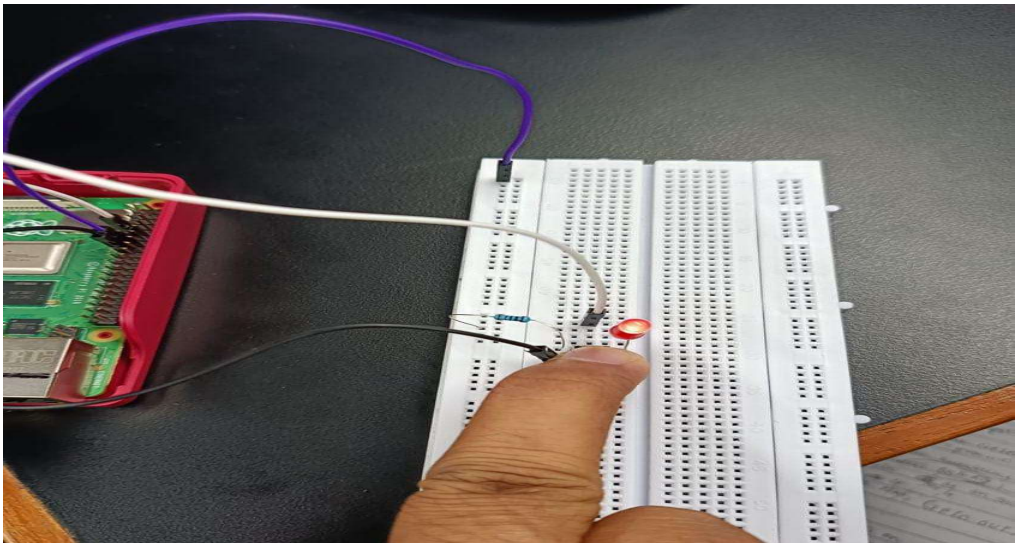
Circuit



When the button is not pressed



When the button is pressed



Discussion:

Conclusion: